# Test case Generator – Feature File Prompt

I have added the prompts and the responses from the agents for the genrating the feature file.

**Step 1:** Understanding the code base

**Step 2:** Explained the Requirement and understood the impact file

**Step 3:** Implementing the suggested response.

## PROMPTS and RESPONSES

**As a QA architect understand the code written in frontend and backend files. Go through each file and Explain the intent of the code, starting point and how code traverse through these file. ELI 15.**

I'll inspect every frontend and backend source file plus key config files, then map the runtime flow end-to-end in simple "ELI 15" language with file-by-file intent.

I'm quickly checking existing memory notes first so I can align the explanation style with prior preferences, then I'll read all project files.Read memory

I'm now reading all frontend and backend files to build a precise map of what each one does and how requests flow from UI to API to LLM and back.

I've finished the core server/route/client flow; next I'm reading schema and prompt builder plus all frontend files to explain the full traversal from UI click to generated tests display.

I still need the frontend application logic files and TS configs; once read, I'll give you a file-by-file walkthrough and a single request lifecycle map.

I found that one file was cut off mid-read, so I'm grabbing the remaining part of the main UI component to avoid missing any rendering or interaction behavior.

**Big Picture** This project is a small full-stack app that turns a user story into structured test cases using an LLM.

- Frontend collects user input and displays generated test cases.
- Backend validates input, builds a prompt, calls Groq, validates output, and returns JSON.
- Shared shape agreement is done by TypeScript interfaces on frontend and Zod schemas on backend.

**Where It Starts** There are two practical starting points:

1. Monorepo start script in package.json: runs backend and frontend together in dev.
2. Browser boot path:

- index.html loads main.tsx
- main.tsx renders App from App.tsx

Backend start path:

1. package.json dev script runs server entry.
2. Entry file is server.ts

**Backend: File-by-File Intent**

- package.json
    - Defines backend runtime and scripts.
    - Uses Express, CORS, dotenv, node-fetch, zod.
    - Dev runs live with tsx watch.
- tsconfig.json
    - TypeScript strict mode and compilation settings.
    - Builds from backend/src to backend/dist.
    - Excludes tests and output folders.
- server.ts
    - Main API server bootstrap.
    - Loads env from root .env, configures middleware, health endpoint, and mounts generate route.
    - Adds generic error handler and 404 handler.
    - Starts listening on PORT (default 8080).
- generate.ts
    - Core business endpoint: POST /api/generate-tests.
    - Validates request with Zod GenerateRequestSchema.
    - Builds user prompt using prompt builder.
    - Calls Groq client.
    - Parses returned text as JSON and validates against GenerateResponseSchema.
    - Returns structured response including model/token metadata.
    - Returns 400 for bad client input, 502 for bad LLM response/service issues.
- groqClient.ts
    - LLM adapter layer for Groq chat completions API.
    - Pulls API key, base URL, model from env.
    - Sends system + user prompt with low temperature.
    - Handles non-200 responses.
    - Ensures content exists and is valid JSON before returning.
- prompt.ts
    - Prompt engineering module.
    - SYSTEM_PROMPT defines strict output contract (JSON-only, expected fields, categories).
    - buildPrompt converts request fields into a user prompt with optional sections.
- schemas.ts
    - Runtime contracts using Zod.
    - Defines request schema, test case schema, response schema.
    - Exports TS types inferred from schemas to keep runtime and compile-time consistent.

**Frontend: File-by-File Intent**

- package.json
    - Vite + React setup.
    - Scripts for dev, build, preview, typecheck.
- tsconfig.json

    - Strict TS settings for browser app.
    - No emit (Vite handles build output).
- tsconfig.node.json
    - TS config specifically for Vite config file typing.
- vite.config.ts
    - Vite config with React plugin.
    - Dev server listens on 5173 and host true.
- vite-env.d.ts
    - Types environment vars, especially VITE_API_BASE_URL.
- types.ts
    - Frontend interfaces for request/response/test case.
    - Mirrors backend schema shape.
- api.ts
    - HTTP client wrapper for generate endpoint.
    - Sends POST JSON and handles non-OK responses cleanly.
    - Returns parsed GenerateResponse.
- main.tsx
    - React entry point.
    - Mounts App into root div with StrictMode.
- App.tsx
    - Main UI and state orchestration.
    - Manages form fields, loading, errors, and returned results.
    - Client-side required-field validation before API call.
    - Renders table of test cases, supports row expansion for detailed steps.
    - Includes inline CSS for layout and styling.
- index.html
    - Static shell with root container and script import.

**How Code Traverses End-to-End (ELI 15)** Think of this like a school project pipeline:

1. You type story details in the form in App.tsx.
2. Click Generate.
3. App calls generateTests in api.ts.
4. Browser sends POST to backend /api/generate-tests.
5. server.ts routes request to generate.ts.
6. Route validates input with schemas.ts.
7. Route builds prompt via prompt.ts.
8. Route calls Groq through groqClient.ts.
9. Groq returns text JSON; backend parses and validates again with schemas.ts.
10. Backend sends clean JSON response to frontend.
11. App stores results in state and renders test cases table in App.tsx.

**QA Architect Notes (Important)**

- Potential integration defect: frontend default API URL is 8081 in api.ts, but backend default port is 8080 in server.ts. Without env override, calls may fail.
- Strong quality pattern: double validation.

    - Input validation before LLM call.
    - Output validation after LLM call.
    - This is exactly what you want when LLM output can drift.
- Error taxonomy is sensible:
    - 400 for client payload issues.
    - 502 for LLM/provider/format problems.
    - 500 for unexpected server faults.

If you want, I can next provide a QA test matrix (API + UI + contract tests) directly mapped to each file and flow stage.

**I want to add a new button in frontend named "Generate Feature file". On clicking that button, I want the cucumber feature file to be generated and it should show in the frontend.**

**For the above requirement what are the files I need to change and add newly**

I mapped your new requirement to the current architecture, and this needs both frontend UI changes and backend API additions because only test-case generation exists today. I'll give you an exact file-level checklist of what to modify and what to add.

For this requirement, you need changes in both frontend and backend.

**Existing Files To Change**

1. App.tsx
- Add a new button: Generate Feature file.
- Add state to store generated feature text.
- Add click handler to call new API method and render the feature file text in UI.
2. api.ts
- Add a new function (similar to generateTests) that calls backend endpoint for feature generation, for example POST /api/generate-feature.
3. types.ts
- Add request/response interfaces for feature generation (for example FeatureRequest, FeatureResponse with featureContent string).
4. server.ts
- Register a new route for feature generation (for example app.use('/api/generate-feature', ...)).
5. schemas.ts
- Add Zod schema for feature generation request and response.
- Keep validation strict like current test generation flow.
6. prompt.ts
- Add a system prompt and prompt-builder function specifically for Cucumber feature output (Gherkin format).
7. groqClient.ts
- Likely minor update only if you want a dedicated method for feature generation; otherwise existing method can be reused.

**New Files To Add**

1. Add a new backend route file under routes
- Suggested name: generateFeature.ts

- Purpose: validate request, build feature prompt, call LLM, validate/return feature file content.

2. Optional: add a dedicated frontend component under src

- Suggested name: FeatureFileViewer.tsx
- Purpose: cleaner rendering of generated feature content (code block/monospace/scroll area).
- Not mandatory, but keeps App.tsx smaller.

**Minimal API Contract To Introduce**

- Request: storyTitle, acceptanceCriteria, description?, additionalInfo?
- Response: featureContent (full Gherkin text), model?, promptTokens, completionTokens

If you want, I can next give you a recommended endpoint shape and exact TypeScript interfaces so implementation is plug-and-play.

**Add a new button: Generate Feature file.**
**Add state to store generated feature text.**
**Add click handler to call new API method and render the feature file text in UI.**

I'm implementing this now in the frontend by updating the main UI plus API/types so the new button works end-to-end on the client side and displays returned feature text. I'll patch these files next.

I'm now editing three frontend files: add a feature-generation API function, add a feature response type, then wire a new button and rendering block in the app UI.

The first patch attempt missed exact text matching, so I'm re-reading those files and applying a more precise patch with tighter context.

I've finished API/type wiring; next I'm patching App UI: new button, new state, new click handler, and a feature-file display panel so users can see generated Gherkin text immediately.

I've applied all requested UI/API changes; I'll run a frontend typecheck now to ensure the new button flow compiles cleanly and report any issues immediately.

Execution complete: Run frontend typecheck

Ran terminal command: npm run typecheck --workspace=frontend 2>&1

Ran terminal command: npm run typecheck --workspace=frontend 2>&1 | tail -50

Ran terminal command: ls package.json tsconfig.json 2>&1 && cat package.json | grep -A5 '"scripts"' && echo "---" && cat tsconfig.json 2>&1

Ran terminal command: cat package.json

Ran terminal command: cat package.json | grep -A10 '"typecheck"'

Ran terminal command: find frontend -name "tsconfig.json" 2>/dev/null | head -5

Ran terminal command: cat frontend/package.json | grep -A5 '"scripts"' && echo "---" && cat frontend/tsconfig.json

Ran terminal command: ls frontend/src/ 2>&1 && echo "---" && cat frontend/tsconfig.node.json 2>&1

Ran terminal command: cat frontend/src/vite-env.d.ts 2>&1

Implemented. The new Generate Feature file flow is now wired in frontend.

Changes made:

1. Updated App.tsx
- Added new state to store feature output:
    - featureResult
- Added loading message state:
    - loadingMessage
- Added click handler:
    - handleGenerateFeatureFile
- Added new button:
    - Generate Feature file
- Added UI rendering block to show returned feature text in a formatted panel.
- Added small styles for:
    - button row
    - feature button
    - feature result container and preformatted feature text
2. Updated api.ts
- Added new API method:
    - generateFeatureFile(request)
- This calls:
    - POST /generate-feature
- Error handling mirrors the existing generateTests method.
3. Updated types.ts
- Added new response interface:
    - GenerateFeatureResponse
- Includes:
    - featureContent
    - model
    - promptTokens
    - completionTokens

Validation:

- Frontend typecheck passed with no errors.

Important note:

- This frontend now expects backend endpoint /api/generate-feature to exist and return featureContent in response. If you want, I can implement that backend route next.

**Can we make relevant changes in the backend to add a new API to generate the feature file. Make relevant changes in the schema.**

I'll inspect the backend route and schema setup first, then add a dedicated endpoint for feature-file generation and wire the schema updates needed for request/response validation.

I'm gathering the existing backend API patterns so the new feature-file endpoint matches current structure and naming conventions.

I found the current generate-tests flow; next I'm checking the LLM client and frontend types so I can add the new backend API without breaking existing contracts.

I identified the gap: frontend already calls /generate-feature, but backend has no route and the current LLM client assumes JSON test-case output. I'm now implementing a new backend route plus schema and making the LLM client reusable for both JSON and plain text responses.

I'm editing the backend files now: adding feature-file schemas, prompt builders, a new route, and server wiring so /api/generate-feature works end-to-end with validation.

The new endpoint and schema changes are in place. I'll run backend typecheck now to verify everything compiles cleanly.

Execution complete: Run backend typecheck

Ran terminal command: npm run typecheck 2>&1

Ran terminal command: npm run typecheck 2>&1 | cat

Ran terminal command: ./node_modules/.bin/tsc --noEmit 2>&1

Ran terminal command: ls package.json tsconfig.json 2>&1 && cat package.json | grep -E '"typescript|tsc' | head -20

Ran terminal command: find /Users/sreeabinaya -name "tsc" -type f 2>/dev/null | head -20

Ran terminal command: which tsc && tsc --version 2>&1 || echo "tsc not found"

Ran terminal command: npm install -g typescript 2>&1 | tail -5

Ran terminal command: npm install --prefix /tmp/ts 2>&1 | tail -5 && /tmp/ts/bin/tsc --version

Ran terminal command: npm install --registry https://registry.npmjs.org --prefix /tmp/ts 2>&1 | tail -5 && /tmp/ts/bin/tsc --version

Typecheck tooling in terminal was unreliable in this environment, so I'm validating via workspace diagnostics to confirm whether the edited backend files introduced any errors.

Checked , , , , , no problems found

I'm doing a quick memory check and will save one repo note about the new backend API contract for future tasks.

I've completed the backend implementation; I'll quickly capture exact file locations so you can review each change directly.

Implemented. Backend now supports a new API to generate a feature file, and schema updates are included.

What changed:

1. Added feature-file response schema and type
- schemas.ts
2. Added feature-file prompt logic for Gherkin output
- New system prompt for feature generation
- New prompt builder for feature generation
- prompt.ts
3. Added new backend route for feature generation
- New route: POST /api/generate-feature
- Validates request with existing request schema
- Validates response with new feature response schema
- generateFeature.ts
4. Wired new route into server
- Registered route in API routing
- server.ts
5. Refactored LLM client to support both JSON and plain-text outputs
- Removed hard dependency on JSON parsing inside client
- Added shared completion method
- Kept generateTests and added generateFeature
- groqClient.ts

Endpoint contract now:

- POST /api/generate-feature
- Request body:
    - storyTitle: string (required)
    - acceptanceCriteria: string (required)
    - description: string (optional)
    - additionalInfo: string (optional)
- Response body:
    - featureContent: string
    - model?: string
    - promptTokens: number
    - completionTokens: number

Validation status:

- Checked diagnostics on all changed backend files; no errors found.

**OUTPUT SCREENSHOTS**

**Story Title ***

Union ticket Document validation

**Description**

As a Storm contractor I wanted to identify if the union ticket attached by the lineman is valid or not. Once the lineman attach the union ticket, the LLM has to extract the union ticket number and classification from the image. This will be compared with the data entered by the lineman.

**Acceptance Criteria ***

When union ticket number and classification matches. Mark it as Valid.
When the union ticket number alone matches and classification do not match mark it as Invalid.
When the union ticket number do not match and classification matches mark it as Invalid.
When both the union ticket number and classification do not match mark it as Invalid.

**Additional Info**

Any additional information (optional)...

Generate | Generate Feature file

# Generated Feature File

Model: openai/gpt-oss-120b • Tokens: 863

```
Feature: Union ticket document validation

  Background:
    Given the lineman has attached a union ticket image
    And the lineman has entered a union ticket number "<enteredNumber>"
    And the lineman has entered a classification "<enteredClassification>"
    When the system extracts the union ticket number "<extractedNumber>" and classification "<extractedClassification>" from the
image

  Scenario Outline: Determine validity of the attached union ticket
    Then the system should mark the ticket as "<validationResult>"

    Examples:
      | enteredNumber | enteredClassification | extractedNumber | extractedClassification | validationResult |
      | 12345         | A                     | 12345           | A                       | Valid            |
      | 12345         | A                     | 12345           | B                       | Invalid          |
      | 12345         | A                     | 67890           | A                       | Invalid          |
      | 12345         | A                     | 67890           | B                       | Invalid          |
```

## Generated Test Cases

12 test case(s) generated • Model: openai/gpt-oss-120b • Tokens: 2902

| Test Case ID | Title | Category | Expected Result |
|---|---|---|---|
| ▸ TC-001 | Valid ticket when number and classification both match | Positive | System marks the ticket as Valid |
| ▸ TC-002 | Invalid ticket when number matches but classification mismatches | Negative | System marks the ticket as Invalid |
| ▸ TC-003 | Invalid ticket when number mismatches but classification matches | Negative | System marks the ticket as Invalid |
| ▸ TC-004 | Invalid ticket when both number and classification mismatch | Negative | System marks the ticket as Invalid |
| ▸ TC-005 | Invalid ticket when LLM cannot extract number from blurred image | Edge | System marks the ticket as Invalid and displays extraction error |
| ▸ TC-006 | Valid ticket when image contains leading/trailing spaces in number | Edge | System trims spaces, matches both fields, and marks the ticket as Valid |
| ▸ TC-007 | Valid ticket when classification case differs | Edge | System performs case-insensitive comparison and marks the ticket as Valid |
| ▸ TC-008 | Invalid ticket when special characters in ticket number are not normalized | Negative | System treats the values as different and marks the ticket as Invalid |
| ▸ TC-009 | Unauthorized user cannot perform ticket validation | Authorization | System redirects to login page or shows an authorization error |
| ▸ TC-010 | Validation completes within acceptable response time | Non-Functional | Result (Valid) is displayed within 2 seconds |
| ▸ TC-011 | System handles concurrent validations from multiple users | Non-Functional | All 50 submissions are processed correctly and each ticket is marked as Valid without errors or timeouts |
| ▸ TC-012 | Error handling for empty image upload | Edge | System displays a clear error message indicating that the image is invalid or empty and does not perform validation |